



GDExtension

C++ dans Godot 4

Guide d'installation complet pour les participants Windows · Linux · macOS

4.x	C++17	SCons	Débutant
GODOT	LANGAGE	BUILD	NIVEAU

Réalisé par : Amani LAMINE
Marwa HASNAOUI

Ce guide vous accompagne pas à pas dans l'installation de l'environnement de développement C++ pour Godot 4. Chaque outil est expliqué avant son installation. Les erreurs courantes rencontrées sous Windows sont traitées en détail.

Vue d'ensemble

Outil	Role	Version
Python 3.x	Requis pour exécuter SCons	3.8+
SCons	Système de compilation (build system)	4.x
Git	Cloner le dépôt godot-cpp depuis GitHub	2.x+
Compilateur C++	MSVC (Win) / GCC (Linux) / Clang (Mac)	C++17
Godot 4	Le moteur de jeu	4.x

WINDOWS

Etape 1 Installer Python

Pourquoi cet outil ? Python

Python est le langage qui fait tourner SCons (le système de compilation). Sans Python, il est impossible de lancer la compilation de godot-cpp. Python fournit aussi pip, le gestionnaire de paquets qui permet d'installer SCons.

Rendez-vous sur python.org et téléchargez Python 3.x (version la plus récente).

Attention : IMPORTANT : cochez 'Add Python to PATH' avant de cliquer sur Install Now. Sans cette option, les commandes python et pip ne seront pas reconnues dans PowerShell.

Vérifiez dans PowerShell (Windows + X > Terminal) :

```
python --version
```

Etape 2 Installer SCons

Pourquoi cet outil ? SCons

SCons est le système de compilation utilisé par godot-cpp. C'est lui qui lit les instructions de compilation, détecte le compilateur installé sur votre machine, et transforme les fichiers .cpp en bibliothèque utilisable par Godot. Il remplace des outils comme CMake ou Make.

Ouvrez PowerShell et tapez :

```
pip install scons
```

Remarque : Si le message affiche 'Requirement already satisfied', SCons est déjà installé. Ce n'est pas une erreur, continuez.

Vérifiez que SCons fonctionne :

```
scons --version
```

Remarque : Si 'scons' n'est pas reconnu dans PowerShell, utilisez cette commande alternative à la place dans toutes les étapes suivantes :

```
python -m SCons --version
```

Etape 3 Installer Git

Pourquoi cet outil ? Git

Git est le système de contrôle de version utilisé pour télécharger (cloner) le dépôt godot-cpp depuis GitHub. Il permet aussi de récupérer le sous-module godot-headers inclus dans godot-cpp. Git est installé en priorité car il sera nécessaire à l'étape 4 pour cloner le dépôt.

```
winget install Git.Git
```

Verifiez :

```
git --version
```

Etape 4 Créer le dossier mon_jeu_2D et cloner godot-cpp

Pourquoi cet outil ? Dossier de projet + godot-cpp

On crée d'abord le dossier racine mon_jeu_2D qui contiendra tout votre projet (votre jeu Godot ET la bibliothèque C++). Ensuite on clone godot-cpp à l'intérieur avec Git : c'est la bibliothèque officielle de bindings C++ pour Godot qui contient toutes les classes de l'API traduite en C++ (Node, Sprite2D, etc.).

Dans PowerShell, créez le dossier de projet et entrez dedans :

```
mkdir C:\Users\<votre_nom>\mon_jeu_2D  
cd C:\Users\<votre_nom>\mon_jeu_2D
```

Clonez godot-cpp dans ce dossier (utilisez Git installé à l'étape 3) :

```
git clone https://github.com/godotengine/godot-cpp  
cd godot-cpp  
git submodule update -init
```

Remarque : La commande 'git submodule update --init' télécharge godot-headers, qui contient les définitions de l'API Godot en C++. Sans cette étape, la compilation échoue.

Après ces commandes, votre structure de dossiers doit ressembler à :

```
mon_jeu_2D\  
  godot-cpp\  
    godot-headers\    <- cree par git submodule  
    src\  
    SConstruct
```

Etape 5 Installer le compilateur C++ (Visual Studio Build Tools)

Pourquoi cet outil ? Compilateur C++ (MSVC)

Le compilateur C++ est l'outil qui traduit votre code source C++ en fichier binaire (.dll) que Godot peut charger. Sans lui, SCons démarre mais échoue immédiatement avec l'erreur 'Le fichier spécifié est introuvable'. MSVC est le compilateur officiel Microsoft, optimisé pour Windows.

Si SCons échoue avec l'erreur 'Le fichier spécifié est introuvable' :

Cause : Visual Studio Build Tools n'est pas installé ou pas détecté.

Solution — Installer les Build Tools :

1. Allez sur : <https://visualstudio.microsoft.com/fr/downloads/>
2. Faites défiler jusqu'à **Outils pour Visual Studio**
3. Téléchargez **Build Tools for Visual Studio 2022**
4. Lancez l'installateur et cochez **Développement Desktop en C++**
5. Cliquez Installer et attendez (environ 5 Go)

Après l'installation — utiliser le terminal de Visual Studio :

Ne lancez pas PowerShell normalement. Utilisez le terminal spécial qui configure

automatiquement le compilateur :

1. Cherchez **Developer Command Prompt for VS 2022** dans le menu Démarrer
2. Naviguez vers votre dossier :

```
| cd C:\Users\<votre_nom>\mon_jeu_2D\godot-cpp
```
3. Relancez la compilation :

```
| python -m SCons platform=windows
```

Alternative legere — MinGW (~500 Mo au lieu de 5 Go) :

Installez d'abord MinGW :

```
| winget install MinGW.MinGW
```

Puis compilez avec :

```
| python -m SCons platform=windows use_mingw=yes
```

Etape 6 Compiler les bindings

Pourquoi cet outil ? Compilation des bindings

Cette étape transforme le code source C++ de godot-cpp en fichier binaire (.lib) que votre propre code C++ pourra utiliser. C'est une opération à faire une seule fois par machine. SCons (étape 2) lit le fichier SConstruct et le compilateur MSVC (étape 4) effectue la compilation.

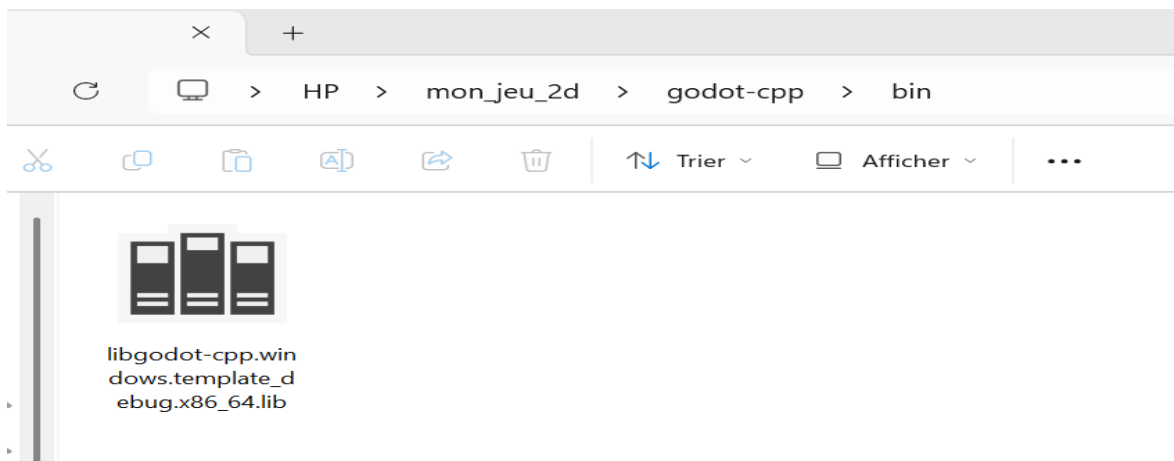
Assurez-vous d'être dans le dossier godot-cpp, puis :

```
python -m SCons platform=windows
```

Ou si scons est dans le PATH :

```
scons platform=windows
```

Succes : La compilation prend 5 a 15 minutes. Un dossier bin/ est créé avec les fichiers .lib. C'est normal de voir défiler beaucoup de lignes de compilation.



LINUX

Sur Linux, tous les outils sont disponibles dans les gestionnaires de paquets officiels. Une seule commande installe tout.

Etape 1 Installer toutes les dépendances

Pourquoi cet outil ? python3 + scons + build-essential + git

python3 fournit l'environnement d'exécution pour SCons. build-essential (ou gcc-c++) installe GCC, le compilateur C++ standard sur Linux. scons est le système de compilation. git permet de cloner godot-cpp.

Ubuntu / Debian

```
sudo apt update
sudo apt install -y python3 python3-pip git build-essential scons
```

Fedora / RHEL

```
sudo dnf install -y python3 python3-pip git gcc-c++ scons
```

Arch Linux

```
sudo pacman -S python python-pip git base-devel scons
```

Etape 2 Verifier les installations

```
python3 --version && scons --version && git --version
```

Etape 3 Creer le dossier mon_jeu_2D et cloner godot-cpp

Pourquoi cet outil ? Dossier de projet + godot-cpp

On crée le dossier racine mon_jeu_2D, puis on clone godot-cpp dedans avec Git. Ce dossier contiendra a terme votre jeu Godot et la bibliothèque C++.

```
mkdir ~/mon_jeu_2D
cd ~/mon_jeu_2D
git clone https://github.com/godotengine/godot-cpp
cd godot-cpp
git submodule update --init
```

Etape 4 Compiler les bindings

```
scons platform=linux
```

Succes : Fichiers. a (archive statique) génères dans bin/. Compilation ~5 min.

macOS

Sur macOS, Homebrew simplifie l'installation de tous les outils. Les puces Apple Silicon (M1/M2/M3) et Intel sont toutes les deux prises en charge.

Étape 1 Installer Xcode Command Line Tools

Pourquoi cet outil ? Xcode Command Line Tools

Ce paquet installe le compilateur Clang (l'équivalent de GCC sur macOS), les en-têtes système, et les outils de base de développement. C'est le compilateur C++ que SCons utilisera pour compiler godot-cpp. Il est gratuit et fourni directement par Apple.

```
xcode-select --install
```

Une fenêtre s'ouvre pour confirmer. Acceptez et attendez la fin (environ 500 Mo).

Étape 2 Installer Homebrew

Pourquoi cet outil ? Homebrew

Homebrew est le gestionnaire de paquets de référence sur macOS. Il permet d'installer Python, SCons et Git en une seule commande, sans configuration manuelle du PATH.

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Étape 3 Installer Python, SCons et Git

Pourquoi cet outil ? python + scons + git

Mêmes rôles que sur les autres systèmes : Python fait tourner SCons, SCons compile les bindings, Git clone le dépôt. Homebrew les installe tous les trois et configure automatiquement le PATH.

```
brew install python scons git
```

Étape 4 Créer le dossier mon_jeu_2D et cloner godot-cpp

Pourquoi cet outil ? Dossier de projet + godot-cpp

On crée le dossier racine mon_jeu_2D, puis on clone godot-cpp dedans. Ce dossier sera la racine de tout votre projet de jeu.

```
mkdir ~/mon_jeu_2D  
cd ~/mon_jeu_2D  
git clone https://github.com/godotengine/godot-cpp  
cd godot-cpp  
git submodule update --init
```

Étape 5 Compiler les bindings

Intel Mac :

```
scons platform=macos arch=x86_64
```

Apple Silicon (M1 / M2 / M3) :

```
scons platform=macos arch=arm64
```

Universal (Intel + Apple Silicon) :

```
scons platform=macos arch=universal
```

Succes : Fichiers .a génères dans bin/. Pour connaitre votre architecture : `uname -m`

Erreur	Cause	Solution
'scons' n'est pas reconnu	SCons absent du PATH	Utiliser python -m SCons
'Le fichier specifié est introuvable'	Compilateur C++ absent	Installer VS Build Tools ou MinGW
'Requirement already satisfied'	SCons déjà installé	Normal — pas une erreur
git submodule vide	Sous-module non initialisé	git submodule update --init
Erreur de permission pip	Python système protège	Ajouter --user ou utiliser venv
'python' non reconnu	Python absent ou sans PATH	Reinstaller Python avec 'Add to PATH'

Etape finale — Tester que tout fonctionne

Cette étape vous permet de vérifier que l'ensemble de la chaîne d'outils fonctionne correctement, du code C++ jusqu'à l'affichage dans Godot. Chaque sous-étape indique quel outil elle sollicite.

Etape T1 Vérifier tous les outils installés

Pourquoi cet outil ? Python + SCons + Git + Compilateur

Cette vérification confirme que chaque outil est accessible depuis le terminal. Si l'une de ces commandes échoue, revenez à l'étape correspondante dans ce guide.

Windows (PowerShell) :

```
python --version
python -m SCons --version
git --version
```

Linux / macOS :

```
python3 --version && scons --version && git --version
```

Succes : Résultat attendu : trois numéros de version s'affichent sans erreur.

Etape T2 Vérifier que bin/ contient les fichiers compilés

Pourquoi cet outil ? SCons + Compilateur C++

Après la compilation (étape 6), le dossier bin/ doit contenir une bibliothèque statique. Son existence confirme que SCons et le compilateur ont fonctionné correctement ensemble.

Windows :

```
dir C:\Users\<votre_nom>\mon_jeu_2D\godot-cpp\bin
```

Linux / macOS :

```
ls ~/mon_jeu_2D/godot-cpp/bin
```

Succes : Résultat attendu : vous voyez un fichier .lib (Windows) ou .a (Linux/macOS). Exemple : libgodot-cpp.windows.template_debug.x86_64.lib

Etape T3 Créer et compiler une extension C++ minimale

Pourquoi cet outil ? Compilateur C++ + SCons + godot-cpp

On crée une extension C++ très simple qui ne fait qu'afficher un message dans la console Godot. C'est le test le plus complet : il valide que votre code C++ se compile correctement et s'interface avec godot-cpp.

Vous pouvez créer vos fichiers avec Visual Studio Code.

Dans le dossier mon_jeu_2D, créez la structure suivante :

```
mon_jeu_2D\
  godot-cpp\          <- déjà clone
  mon_extension\
    src\
      hello.hpp
      hello.cpp
      register_types.cpp
      register_types.cpp
    SConstruct
```

Fichier src/hello.hpp

```
#pragma once
#include <godot_cpp/classes/node.hpp>
#include <C:/Users/HP/mon_jeu_2d/godot-cpp/src/core/class_db.hpp>

namespace godot {

class Hello : public Node {
    GDCLASS(Hello, Node)

protected:
    static void _bind_methods();

public:
    void _ready();
};
```

Fichier src/hello.cpp

```
#include "hello.hpp"
#include <godot_cpp/variant/utility_functions.hpp>

using namespace godot;

void Hello::_bind_methods() {}

void Hello::_ready() {
    UtilityFunctions::print("Hello from C++ !");
}
```

Fichier src/register_types.hpp

```
#pragma once

#include <godot_cpp/core/class_db.hpp>

using namespace godot;

void initialize_mon_extension_module(ModuleInitializationLevel p_level);
void uninitialize_mon_extension_module(ModuleInitializationLevel p_level);
```

Fichier src/register_types.cpp

```
#include "register_types.hpp"
#include "hello.hpp"
#include <gdextension_interface.h>
#include <godot_cpp/core/defs.hpp>
#include <godot_cpp/godot.hpp>

using namespace godot;

void initialize_hello(ModuleInitializationLevel p_level) {
    if (p_level != MODULE_INITIALIZATION_LEVEL_SCENE) return;
    ClassDB::register_class<Hello>();
}
```

```
void uninitialized_hello(ModuleInitializationLevel p_level) {}

extern "C" {
    GDEExtensionBool GDE_EXPORT hello_init(
        GDEExtensionInterfaceGetProcAddress p_get_proc_address,
        const GDEExtensionClassLibraryPtr p_library,
        GDEExtensionInitialization *r_initialization) {
        GDEExtensionBinding::InitObject init_obj(
            p_get_proc_address, p_library, r_initialization);
        init_obj.register_initializer(initialize_hello);
        init_obj.register_terminator(uninitialize_hello);
        init_obj.set_minimum_library_initialization_level(
            MODULE_INITIALIZATION_LEVEL_SCENE);
        return init_obj.init();
    }
}
```

Fichier SConstruct (à la racine de mon_extension)

```
env = SConscript('../godot-cpp/SConstruct')
env.Append(CPPPATH=['src/'])
sources = Glob('src/*.cpp')

if env['platform'] == 'windows':
    library = env.SharedLibrary('bin/hello', source=sources)
else:
    library = env.SharedLibrary('bin/libhello', source=sources)
Default(library)
```

Compiler l'extension (utilise SCons + Compilateur)

Windows :

```
cd C:\Users\<votre_nom>\mon_jeu_2D\mon_extension
python -m SCons platform=windows
```

Linux :

```
cd ~/mon_jeu_2D/mon_extension && scons platform=linux
```

macOS :

```
cd ~/mon_jeu_2D/mon_extension && scons platform=macos arch=arm64
```

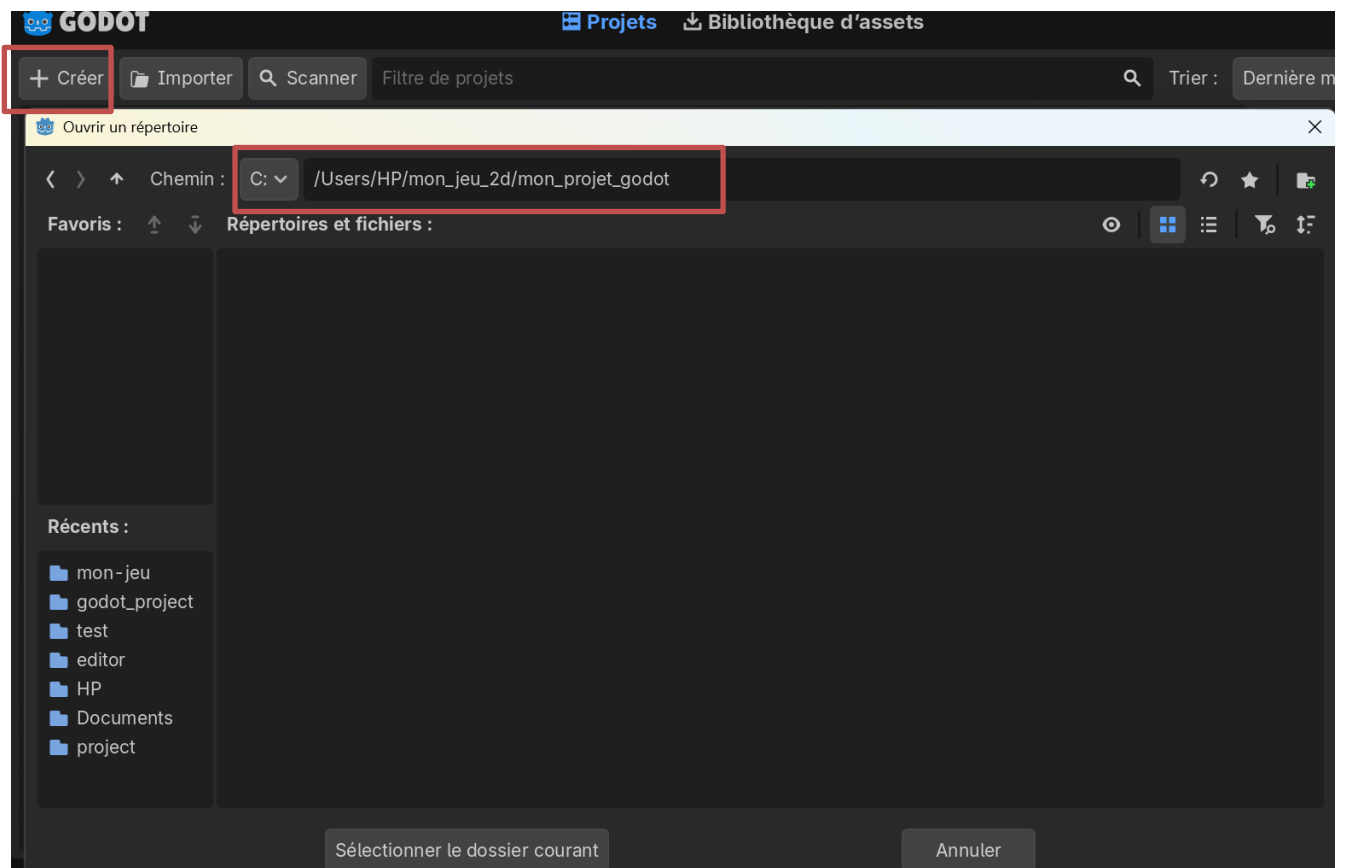
Succes : Résultat attendu : un fichier hello.dll (Windows) ou libhello.so (Linux) ou libhello.dylib (macOS) apparaît dans mon_extension/bin/.

Etape T4 Tester dans Godot

Pourquoi cet outil ? Godot 4

Cette étape finale charge votre extension dans Godot et confirme que tout fonctionne de bout en bout. On crée un projet Godot minimal, on y ajoute le fichier .gdextension qui pointe vers votre .dll/.so, et on attache votre node Hello à une scène.

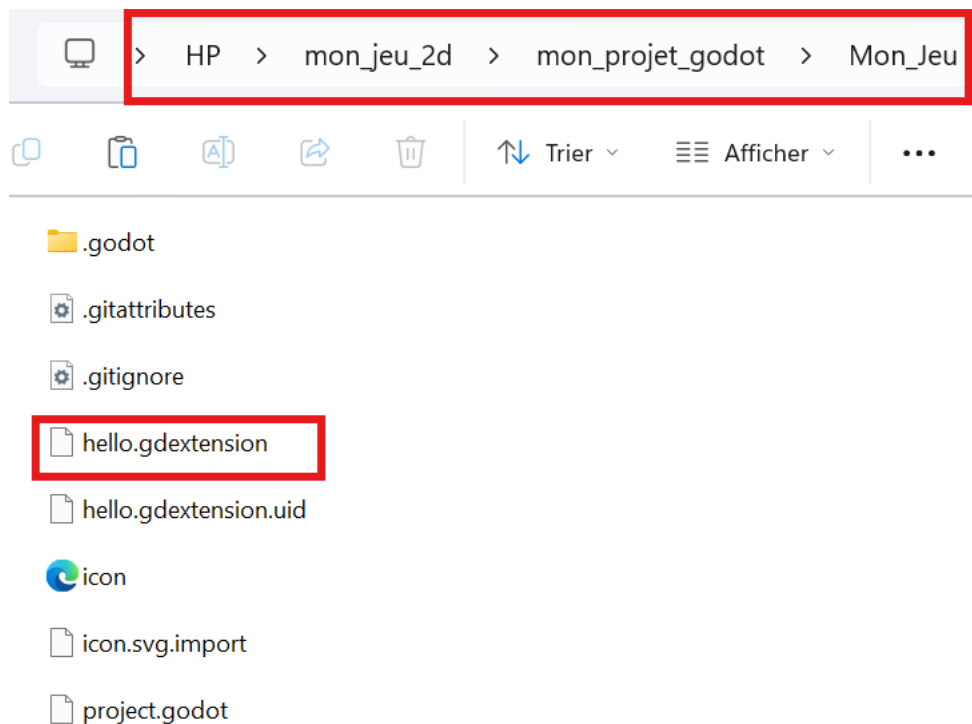
1. Installer Télécharge **Godot 4.6** depuis le site officiel : <https://godotengine.org/download>
2. Créez un nouveau projet dans mon_jeu_2D/mon_projet_godot/
2. Ouvrez Godot Engine 4, puis créez un nouveau projet comme suit :



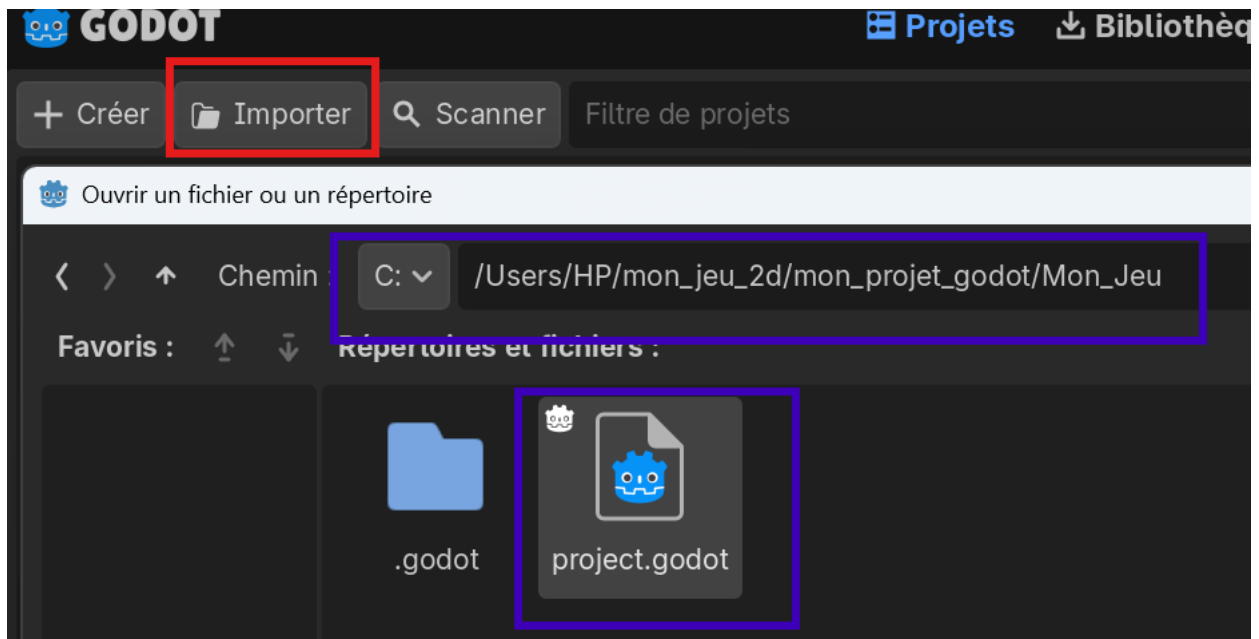
3. Fermez le Godot et dans ce dossier **Mon_jeu**, créez le fichier **hello.gdextension** :

```
[configuration]
entry_symbol = "hello_init"
compatibility_minimum = "4.1"

[libraries]
windows.debug.x86_64 =
"C:/Users/HP/mon_jeu_2d/mon_extension/bin/windows/mon_extension.windows.template_debug.x86_64.dll"
```

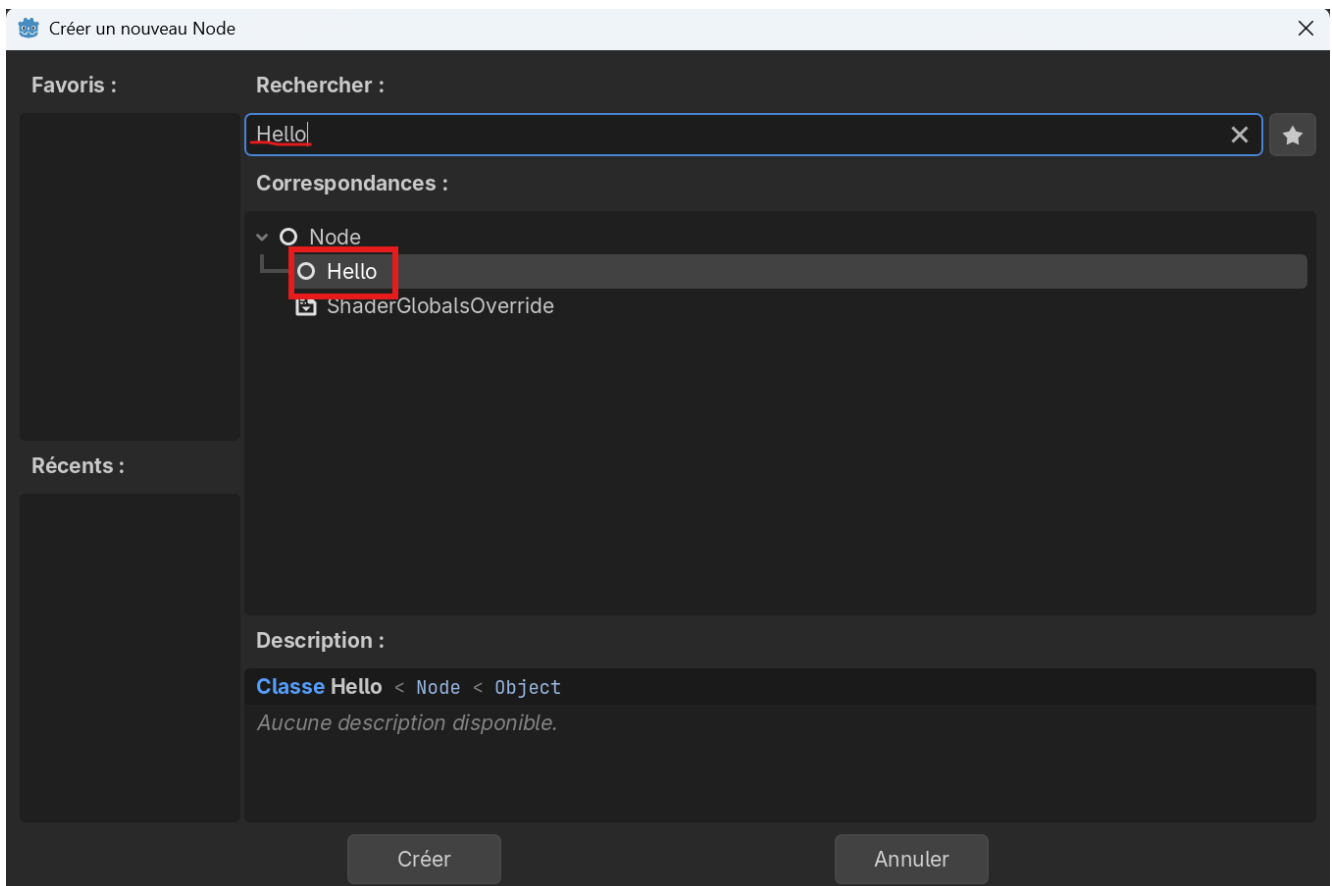


4. Relancez Godot et importer le projet déjà créer



5. Créer une scène et ajouter votre node

- Scene
- Cliquez Node2D(puis le bouton +)
- Dans la recherche, tapez Hello → votre node custom doit apparaître dans la liste
- Sélectionnez-le → Créer
- Sauvegardez la scène (Ctrl+S) → main.tscn



Succes : Résultat attendu : le message 'Hello depuis C++ !' s'affiche dans la console de Godot. Votre environnement C++ est entièrement fonctionnel !

```
Godot Engine v4.6.2.stable.official (c) 2007-present Juan Linietsky, Ariel
Manzur & Godot Contributors.
--- Debug adapter server started on port 6006 ---
--- GDScript language server started on port 6005 ---
Nouvelle racine de scène
Hello from C++ !
Créer un nœud
```